

Security Policy — pqc_crypto_module v0.1.0

Disclaimer: This module is prepared for FIPS 140-3 evaluation and is not currently validated. This document is structured to align with FIPS 140-3 Security Policy requirements (NIST IG 7.1) but has not been reviewed by a CMVP-accredited laboratory.

1. Module Name and Identification

- **Module name:** pqc_crypto_module
- **Version:** 0.1.0
- **Type:** Software cryptographic module
- **Security level target:** FIPS 140-3 Level 1 (software only)
- **Description:** A Rust-based post-quantum cryptographic module providing digital signature, key encapsulation, and hashing services for the Cerulean Ledger distributed ledger platform.

2. Cryptographic Boundary

The cryptographic boundary encompasses all source files within crates/pqc_crypto_module/src/:

File	Responsibility
api.rs	Single public entry point for all approved operations
mldsa.rs	ML-DSA-65 key generation, signing, verification
mlkem.rs	ML-KEM-768 key encapsulation (FIPS 203)
hashing.rs	SHA3-256 hashing
rng.rs	CSPRNG wrapper with continuous test
self_tests.rs	Known Answer Tests (KATs)
approved_mode.rs	State machine and approved-mode enforcement
types.rs	Cryptographic types with zeroization
errors.rs	Error types
legacy.rs	Non-approved algorithms (outside approved boundary)

File	Responsibility
lib.rs	Module re-exports

The boundary is enforced by the Rust crate system. External code accesses approved cryptographic operations through `pqc_crypto_module::api` (state-guarded) or `pqc_crypto_module::mldsa/mlkem/slhdsa` (internal modules with `require_approved()` guards). Non-approved legacy algorithms (Ed25519, SHA-256) are accessed through `pqc_crypto_module::legacy`, which is blocked in Approved state and excluded entirely under `--features approved-only`. Boundary integrity is verified by `tests/api_boundary.rs` and `tests/crypto_boundary`.

Files outside `src/` (tests, Cargo.toml, documentation) are outside the cryptographic boundary.

3. Approved Algorithms

Algorithm	Standard	Purpose	Key Sizes	Output Sizes
ML-DSA-65	FIPS 204	Digital signatures	PK: 1952 B, SK: 4032 B	Sig: 3309 B
ML-KEM-768	FIPS 203	Key encapsulation	PK: 1184 B, SK: 2400 B, CT: 1088 B	SS: 32 B
SHA3-256	FIPS 202	Hashing	N/A	32 B

Implementation note: ML-KEM-768 is implemented via the `pqcrypto-mlkem` crate (v0.1.1), which wraps the reference C implementation of FIPS 203. Encapsulation produces a ciphertext and shared secret; decapsulation deterministically recovers the same shared secret. Invalid ciphertexts are handled via implicit rejection (different shared secret) or error return.

4. Non-Approved Algorithms

The following algorithms are present for backward compatibility with pre-PQC ledger data. They are **not part of the approved cryptographic boundary**.

Algorithm	Purpose	Gating Mechanism
Ed25519	Legacy signature verification	<code>ensure_not_approved()</code> runtime guard
SHA-256	Legacy block hashing	<code>ensure_not_approved()</code> runtime guard
HMAC-SHA256	Legacy MAC operations	<code>ensure_not_approved()</code> runtime guard

When the module is in Approved state, all non-approved algorithm calls return `CryptoError::NonApprovedAlgorithm`. The `approved-only` Cargo feature excludes the legacy module entirely at compile time via `compile_error!`.

See [NON_APPROVED_USAGE.md](#) for details.

5. Roles and Authentication

Role	Description	Authentication
Crypto Officer (CO)	Initializes the module by calling <code>initialize_approved_mode()</code>	Implicit: first caller at process startup
User	Calls approved cryptographic services (sign, verify, hash, encapsulate, decapsulate)	Module state check: <code>require_approved()</code> guard

Both roles require the module to be in Approved state before any cryptographic service is available. There is no password-based or identity-based authentication at the module level; authentication is delegated to the DLT application layer (mTLS + ACL).

6. Services

Approved-mode services (available only in Approved state)

Service	API Function	Description
Module initialization	<code>initialize_approved_mode()</code>	Run self-tests, transition to Approved
ML-DSA key generation	<code>generate_mldsa_keypair()</code>	Generate ML-DSA-65 keypair
ML-DSA signing	<code>sign_message(sk, msg)</code>	Sign a message
ML-DSA verification	<code>verify_signature(pk, msg, sig)</code>	Verify a signature
SHA3-256 hashing	<code>sha3_256(data)</code>	Compute SHA3-256 digest
ML-KEM key generation	<code>generate_mlkem_keypair()</code>	Generate ML-KEM-768 keypair
ML-KEM encapsulation	<code>mlkem_encapsulate(pk)</code>	Encapsulate shared secret
ML-KEM decapsulation	<code>mlkem_decapsulate(sk, ct)</code>	Decapsulate shared secret
Random byte generation	<code>random_bytes(n)</code>	Generate n cryptographically secure random bytes

Non-approved services (blocked in Approved state)

Service	API Function
Legacy Ed25519 sign	<code>legacy_ed25519_sign(sk, msg)</code>

Service	API Function
Legacy Ed25519 verify	legacy_ed25519_verify(pk, msg, sig)
Legacy SHA-256	legacy_sha256(data)
Legacy HMAC-SHA256	legacy_hmac_sha256(key, data)

7. Finite State Model

The module operates as a four-state machine managed by an AtomicU8 with SeqCst ordering:

```

Uninitialized —[initialize_approved_mode()]—> SelfTesting
SelfTesting   —[all KATs pass]—————> Approved
SelfTesting   —[any KAT fails]—————> Error

```

- **Uninitialized (0):** Initial state. All approved operations return `ModuleNotInitialized`.
- **SelfTesting (1):** Transient state during KAT execution.
- **Approved (2):** Operational state. All approved services are available.
- **Error (3):** Terminal state. All operations return `ModuleInErrorState`. Recovery requires process restart.

Forbidden transitions: Error to any other state; Approved to Uninitialized; Uninitialized directly to Approved.

See [FINITE_STATE_MODEL.md](#) for the complete model.

8. Physical Security

This is a software-only module. No physical security mechanisms are claimed. The module operates within the physical security perimeter of the host operating system and hardware.

9. Operational Environment

- **Operating system:** Linux (x86_64, aarch64) or macOS (aarch64)
- **Runtime:** Single-process, multi-threaded Rust application
- **Randomness source:** OS-backed CSPRNG via `OsRng` (backed by `getrandom` syscall)
- **Compiler:** Rust nightly toolchain (required for `#![feature(unsigned_is_multiple_of)]` in the parent workspace; the module itself uses stable Rust features)

The module assumes a single-operator environment where the operating system provides process isolation and memory protection.

10. Key Management

Key types

Type	Size	Zeroization	Purpose
MldsPrivateKey	4032 B	ZeroizeOnDrop	ML-DSA-65 signing
MldsPublicKey	1952 B	N/A (public)	ML-DSA-65 verification
MlKemPrivateKey	2400 B	ZeroizeOnDrop	ML-KEM-768 decapsulation
MlKemPublicKey	1184 B	N/A (public)	ML-KEM-768 encapsulation
MlKemSharedSecret	32 B	ZeroizeOnDrop	Shared secret material

Key lifecycle

- **Generation:** Keys are generated inside the module using approved algorithms and OS-backed CSPRNG.
- **Storage:** Keys exist only in process memory. The module does not persist keys to disk.
- **Usage:** Keys are used exclusively through the approved API functions.
- **Destruction:** Private keys and shared secrets implement ZeroizeOnDrop. Memory is overwritten with zeros when the containing variable is dropped.

See [KEY_MANAGEMENT.md](#) for the complete key management policy.

11. Self-Tests

Self-tests run during `initialize_approved_mode()` before any cryptographic service becomes available.

Test	Algorithm	Method
KAT SHA3-256	SHA3-256	Hash empty string, compare to known digest
KAT ML-DSA-65	ML-DSA-65	Generate keypair, sign, verify, corrupt signature, verify rejection
KAT ML-KEM	ML-KEM-768	Generate keypair, encapsulate, decapsulate
Continuous RNG test	OsRng	Generate two 32-byte outputs, verify they differ

If any test fails, the module transitions to Error state. All subsequent operations are rejected. The module cannot be re-initialized; the process must be restarted.

See [SELF_TEST_DOCUMENTATION.md](#) for the complete self-test specification.

12. Mitigation of Other Attacks

Attack vector	Mitigation
Side-channel timing	ML-DSA and ML-KEM implementations from pqcrypto use constant-time reference code
Memory disclosure	Private keys and shared secrets implement ZeroizeOnDrop
Algorithm downgrade	Runtime guard (ensure_not_approved()) + compile-time exclusion (approved-only feature)
State manipulation	AtomicU8 with SeqCst ordering; Error state is terminal
RNG failure	Continuous RNG test at startup; explicit error propagation on OsRng failure

13. Error State and Recovery

The Error state is **terminal**. Once entered, no cryptographic operations can be performed and no transition to any other state is possible.

Recovery procedure: 1. The Crypto Officer observes that all API calls return `CryptoError::ModuleInErrorState`. 2. The Crypto Officer must terminate the host process. 3. The Crypto Officer restarts the process. The module re-enters Uninitialized state. 4. `initialize_approved_mode()` is called again. If self-tests pass, the module transitions to Approved. 5. If self-tests fail again, the Crypto Officer must investigate the root cause (corrupted binary, hardware fault, or dependency issue) before retrying.

Operator indicators: - `CryptoError::ModuleInErrorState` on any API call → module is in Error state. - `CryptoError::ModuleNotInitialized` → module has not been initialized (call `initialize_approved_mode()`). - `CryptoError::SelfTestFailed(msg)` → initialization failed; module transitioned to Error.

There is no in-process re-initialization path. This is by design: a self-test failure may indicate a compromised binary or hardware fault, and continuing operation would violate fail-closed semantics.

14. Crypto Officer Procedures

Module initialization

1. At process startup, call `pqc_crypto_module::api::initialize_approved_mode()`.
2. If the call returns `Ok()`, the module is in Approved state and all services are available.
3. If the call returns `Err(SelfTestFailed())`, the module is in Error state. Terminate the process, investigate, and restart.
4. Do not attempt to call `initialize_approved_mode()` more than once per process lifetime.

Monitoring

- All API functions return `Result<T, CryptoError>`. The Crypto Officer should log and alert on any `Err` variant.
- Periodic health checks: call `sha3_256(b"healthcheck")` and verify a successful result.

Incident response

- If any operation returns `ModuleInErrorState`, treat the process as compromised and restart immediately.
- If signature verification fails unexpectedly, investigate key compromise before retrying.

15. User Guide

For DLT application developers

- Import only `pqc_crypto_module::api`. Do not import internal modules.
- Call `initialize_approved_mode()` exactly once at process startup before any crypto operation.
- Handle all `Result` errors explicitly. Do not `unwrap()` in production.
- Private keys (`MlDsaPrivateKey`, `MlKemPrivateKey`) are `ZeroizeOnDrop`. Let them drop naturally or call `drop()` explicitly when done.
- Shared secrets (`MlKemSharedSecret`) are also `ZeroizeOnDrop`. Derive session keys or MACs promptly, then drop.

For legacy compatibility

- Use `pqc_crypto_module::legacy::*` only for verifying pre-PQC data.
- Legacy functions are blocked when the module is in Approved state.
- Plan migration to ML-DSA-65 and ML-KEM-768 for all new operations.

16. ACVP / CAVP Test Vector Coverage

The module includes test suites verified against official NIST and community-standard test vectors:

NIST ACVP (Official — NIST ACVP-Server repository)

Algorithm	ACVP Test	Status	Test File
ML-DSA-65	keyGen (seed → pk/sk)	✓ 3 vectors, byte-exact match	tests/acvp_keygen.rs
ML-DSA-65	sigVer (pk + msg + sig → valid/invalid)	✓ 10 vectors (Wycheproof)	tests/nist_kat_vectors.rs
ML-DSA-65	sigGen (internal mode, derand)	✓ 3 vectors, byte-exact match	tests/acvp_full.rs
ML-KEM-768	keyGen (d,z → ek/dk)	✓ 3 vectors, byte-exact match	tests/acvp_full.rs
ML-KEM-768	encapDecap (m → ct/ss)	✓ 3 vectors, byte-exact match	tests/acvp_full.rs
SHA3-256	KAT	✓ 3 FIPS 202 vectors	tests/pqc_gauntlet.rs

C2SP Wycheproof (Community — widely used by crypto libraries)

Algorithm	Test Type	Vectors	Categories
ML-DSA-65	Signature verification	10	Valid baseline, ModifiedSignature, InvalidContext, InvalidHintsEncoding, IncorrectSignatureLength, wrong pk size

Internal Gauntlet (beyond any blockchain DLT)

Category	Tests	What it proves
FIPS 204/203 parameter conformance	6	pk/sk/sig/ct/ss sizes match spec tables
Randomized signing proof	1	ML-DSA non-deterministic per §5.2
Bit-level corruption sweep	2	100+ byte positions, 9 boundary×flip combos
Cross-keypair forgery	2	90 cross-checks across 10 keypairs
Signature malleability	1	complement/reverse/extend attacks

Category	Tests	What it proves
Pathological inputs	5	empty, 1-byte, 1MB, all-zeros, all-ones
Key validation	4	wrong sizes, all-zero keys, truncation
ML-KEM-768 IND-CCA2	5	implicit rejection, cross-keypair, randomization
Entropy chi-squared	3	byte distribution, no duplicates, consecutive differ
Timing baseline	1	valid vs invalid within 10x ratio
Message sensitivity	2	adjacent messages, null-byte injection
Stress cycles	2	100× keygen+sign+verify, 100× encaps+decaps

Total: 133 tests across 12 suites in `pqcrypto_module`.

Deterministic Keygen Infrastructure

The module includes a C FFI binding (`csrc/keypair_from_seed.c`) that implements FIPS 204 §5.1 ML-DSA.`KeyGen_internal(ξ)` — identical to PQClean’s `crypto_sign_keypair()` but with the seed passed as a parameter instead of generated internally. This enables NIST ACVP Known Answer Testing and produces byte-exact output matching the NIST ACVP-Server published expected results.

17. Future Validation Notes

The following items are identified for resolution before formal CMVP submission:

1. **ML-KEM-768**: RESOLVED — implemented via `pqcrypto-mlkem v0.1.1` with roundtrip verification.
2. **ACVP keygen vectors**: RESOLVED — 3 official NIST vectors, byte-exact match via `keypair_from_seed` FFI.
3. **DRBG**: The module delegates randomness to the OS CSPRNG via `getrandom` (Linux: CRNG backed by ChaCha20 since 4.8, CTR_DRBG in FIPS mode; macOS: `SecRandomCopyBytes` backed by Fortuna/CTR_DRBG). For FIPS 140-3 Level 1, this is acceptable when the operational environment runs on a FIPS-validated OS kernel (e.g., RHEL 9 with `fips=1`). A standalone SP 800-90A DRBG wrapper is not required at Level 1 for software-only modules but may be required at Level 2+. Decision: **defer standalone DRBG** until lab feedback; document OS entropy delegation.
4. **Entropy source**: Randomness is sourced from `getrandom(2)` (Linux) or `SecRandomCopyBytes` (macOS). SP 800-90B compliance is inherited from the OS kernel’s entropy subsystem. The module performs a continuous RNG test

(SP 800-90B §4.3) at startup and rejects identical consecutive outputs.

Documented in `rng.rs`.

5. **Physical boundary:** Not applicable (software module), but the operational environment documentation may need expansion for the lab.
6. **Algorithm certificates:** Obtain CAVP algorithm certificates for ML-DSA-65, ML-KEM-768, and SHA3-256 once implementations are validated.
7. **Conditional self-tests:** Add pair-wise consistency tests for key generation if required by the lab.
8. ~~ML-KEM shared secret verification:~~ RESOLVED — KAT self-test now verifies shared secret equality between encapsulate and decapsulate.
9. ~~ACVP sigGen:~~ RESOLVED — `sign_internal_derand` FFI, 3 vectors byte-exact match.
10. ~~ACVP ML-KEM:~~ RESOLVED — `keypair_derand` + `enc_derand` FFI, 6 vectors byte-exact match.